Final Assignment
Numerical Linear Algebra

Spring 2026

361.2.1180

Etai Wigman

Table of Contents

Background 3
Part I – Preliminaries 4
Part II – Fast Rank Estimation 13
Part III – Fast LR Approximation 21
Part IV – Hierarchical Matrix Compression 32
Appendix 40

Background

In this exercise, we study the ability of various algebraic structures to represent efficiently a many-body wave problem of interaction between wave scattering particles.

We are given a 2D grid of dimensions WxW, where $W = \alpha\lambda$, centered at the origin with particles inside placed on a uniform cartesian grid with spacing of $\Delta = \frac{\lambda}{10}$. We denote $\boldsymbol{r}_m = (x_m, y_m)$ the location of the mth particle.

Each particle is influenced by itself and all other particles. We will examine the interaction matrix A and our ability to compress it using various representations and its dependence on the interaction as represented by the kernel function K. The entries of A are given by:

$$A_{mn} = K(\boldsymbol{r}_m - \boldsymbol{r}_n)$$

Part I – Preliminaries

In this part we look at submatrices of matrix A, which means we take two subsets of particles from the grid, source particles and observer particles and denote the interaction matrix between them as A^{os}. We then examine its Singular Value Decomposition (SVD) for two different kernel functions K.

Section (a):

In this section we implement a routine that provided $w, \boldsymbol{r}_c^s, \boldsymbol{r}_c^o$ it constructs the blocks of A^{os} using the following kernel function:

$$K(\boldsymbol{r}_m - \boldsymbol{r}_n) = \begin{cases} \dfrac{1}{|\boldsymbol{r}_m - \boldsymbol{r}_n|^{1/2}} & m \neq n \\ \dfrac{1}{2} & m = n \end{cases}$$

With the following parameters: $\lambda = 1, \alpha = 4, w = \frac{W}{8}$.

The grid of the observer and source particles are as follows:

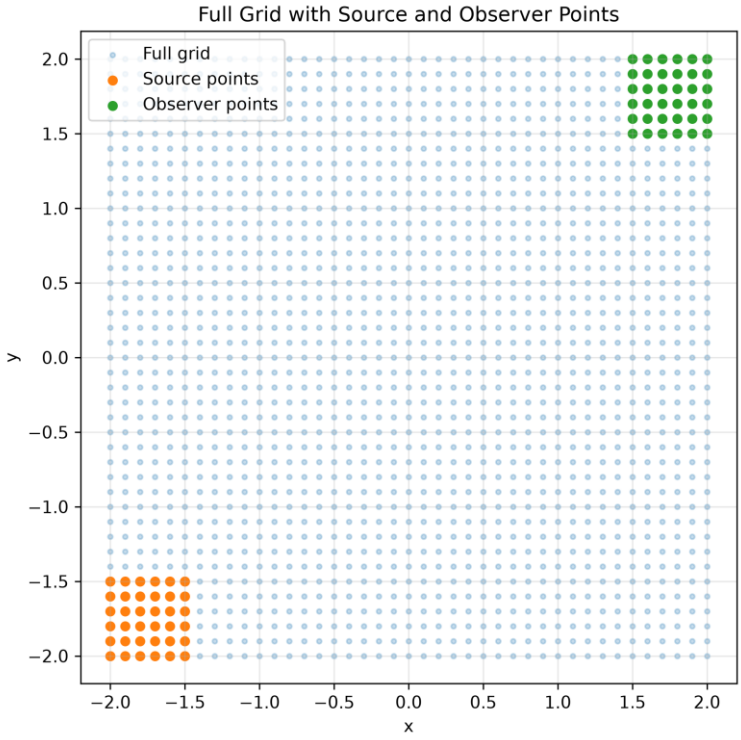


Figure 1 - Source and Observer particles on grid

Using these parameters and the kernel function we obtained the following interaction matrix A^{os}:

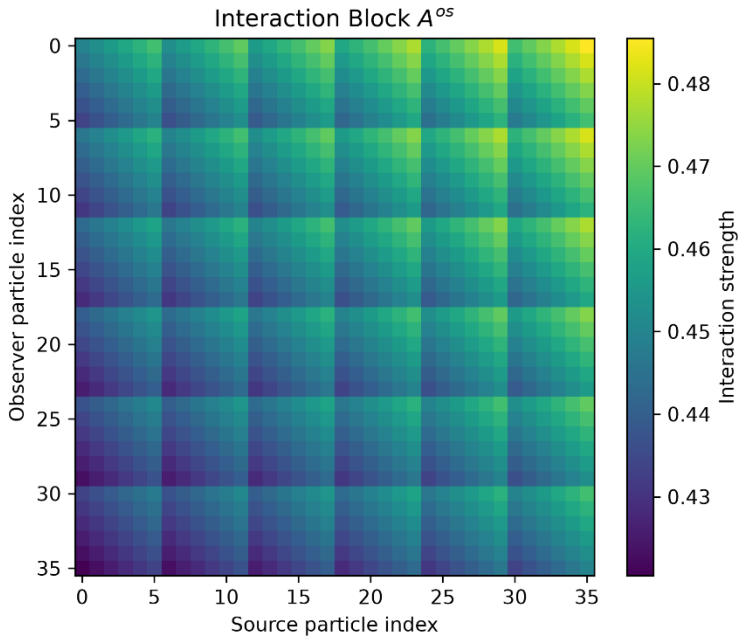


Figure 2 - Interaction block A^{os}

The rows of A^{os} correspond to observer particles, and the columns correspond to source particles.

We computed A^{os} singular values (SV) and its SV curve:

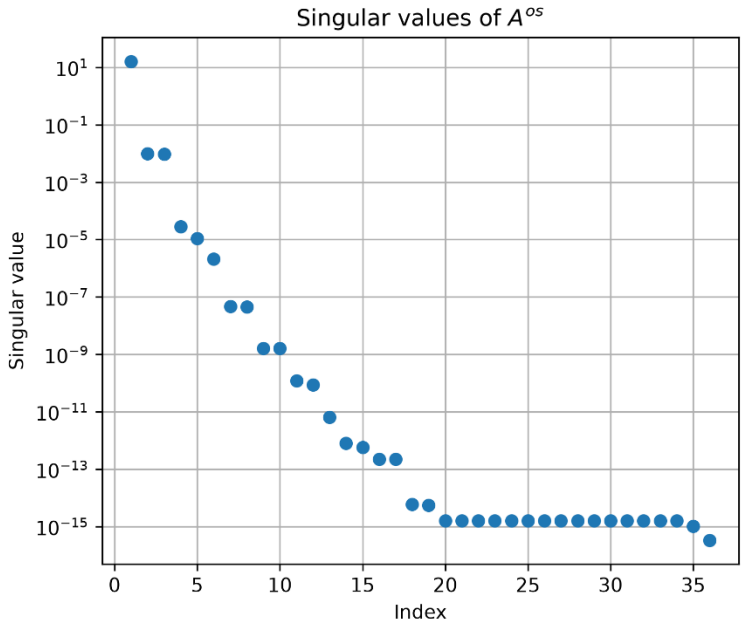


Figure 3 - Singular value curve of interaction block A^{os}

Looking at the kernel function, we can see that the further away the interacted particles, the smaller the interaction strength, now as we observe both the particles grid and matrix A^{os}, we can see that because the distance is large between the subsets and the relative distance between the particles inside the source block to the particles inside the observer block is close, the interaction strength is around the same level.

In addition to that we can see that because the kernel function is smooth, A^{os} is also smooth and does not have sudden changes between close interactions.

As for the SV curve, the rapid decay indicates that the matrix can be approximated well by a low-rank matrix.

Section (b):

In this section we double W (by doubling α) until A^{os} is too large and run section (a) for each configuration.

We show a sequence of SV curves for the different sizes of W:

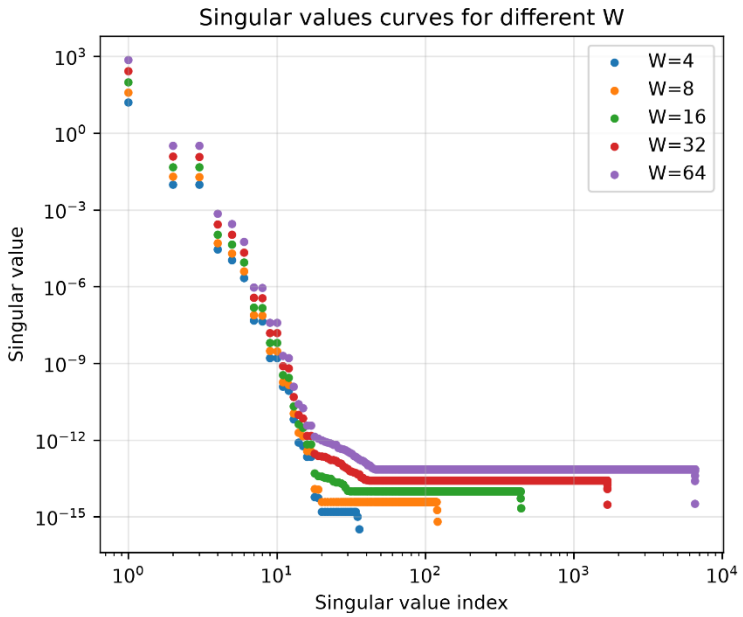


Figure 4 - Singular values curves across different A^{os} sizes

And if we look at the relative SV curves, we obtain the following:

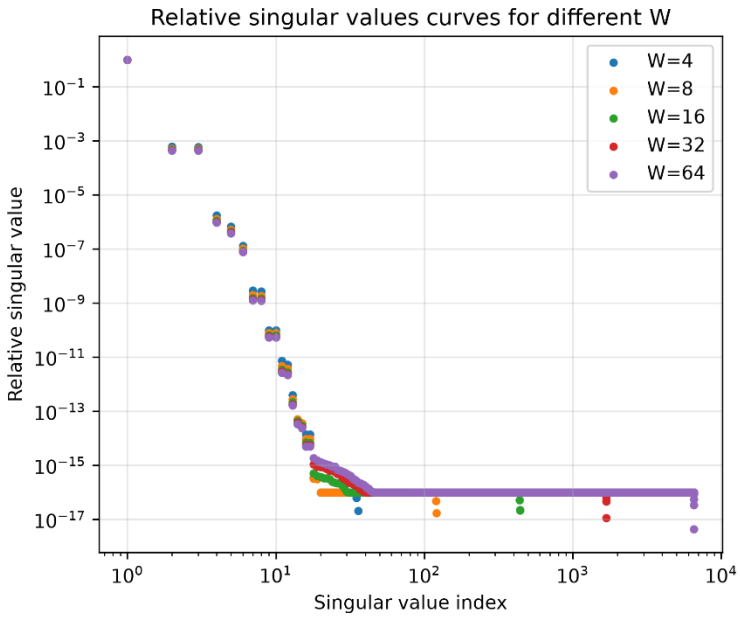


Figure 5 - Relative singular values curves across different A^{os} sizes

We can see that the relative SV across the different sizes remains almost the same for the significant SVs, this means that the matrices structure remains almost the same across the different sizes even though the absolute value of the SV changes slightly across the different sizes.

Next, we plot the rank as a function of W for different relative thresholds:

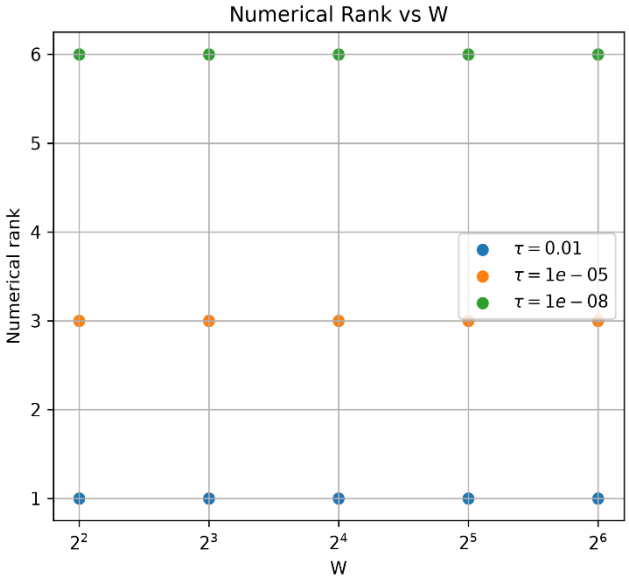


Figure 6 - Numerical rank vs W for different thresholds

The numerical rank was computed using the following relative threshold criterion:

$$R(\tau) = \#\left\{i : \frac{\sigma_i}{\sigma_1} > \tau\right\}$$

This corresponds to using a relative spectral-norm threshold, since the truncated SVD error in the 2-norm is controlled by the first neglected singular value.

We can see that the numerical rank remains the same for the different thresholds across W, we could predict it from seeing the SV curves and the similar structure across the different sizes.

In addition, we plot the computational time as a function of W:

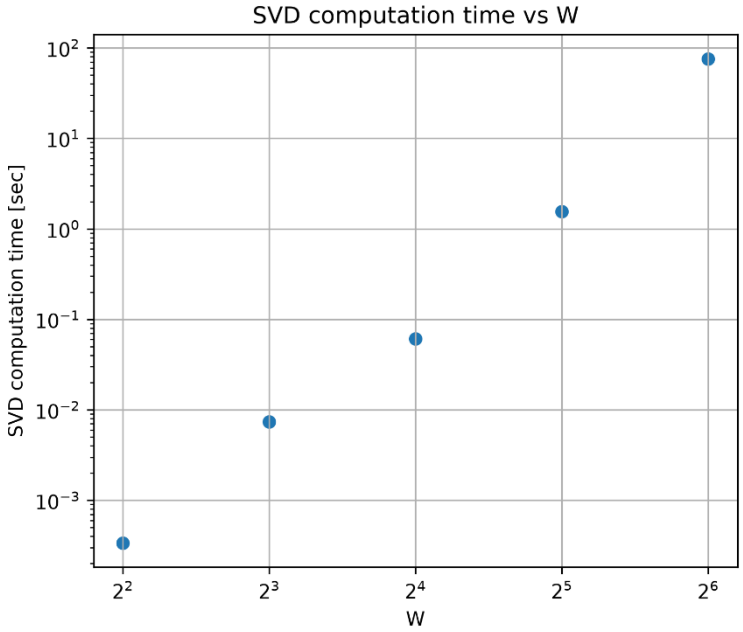


Figure 7 - SVD computation time vs W [sec]

Since W is doubled, the number of particles in each source/observer block grows rapidly. The full SVD is expensive for dense matrices, so the computation time increases sharply as W grows.

Section (c):

For this section we repeat section (b) with a different kernel:

$$K(\boldsymbol{r}_m - \boldsymbol{r}_n) = \begin{cases} \dfrac{e^{-jk|\boldsymbol{r}_m - \boldsymbol{r}_n|}}{|\boldsymbol{r}_m - \boldsymbol{r}_n|^{1/2}} & m \neq n \\ \dfrac{1}{2} & m = n \end{cases}$$

Where $k = \frac{2\pi}{\lambda}$.

We can see that this kernel is similar to the first kernel, but it adds oscillations corresponding to the distance between the observer and source particles.

Now, since we add a more complex structure to the kernel function, we expect the singular values to decay more slowly, so more singular values are needed to represent the matrix to the same tolerance.

Figures 8-11 are the respective Figures 4-7 for the new kernel function:

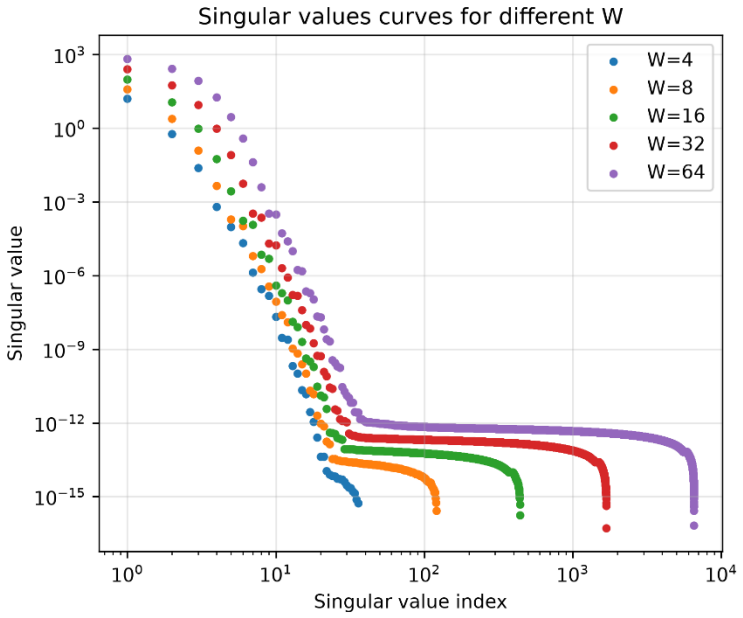


Figure 8 - Singular values curves across different A^{os} sizes

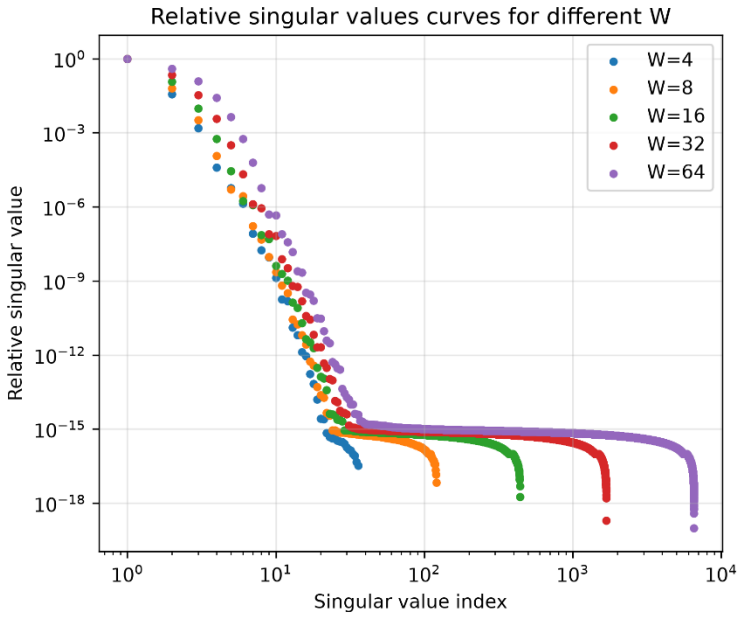


Figure 9 - Relative singular values curves across different A^{os} sizes

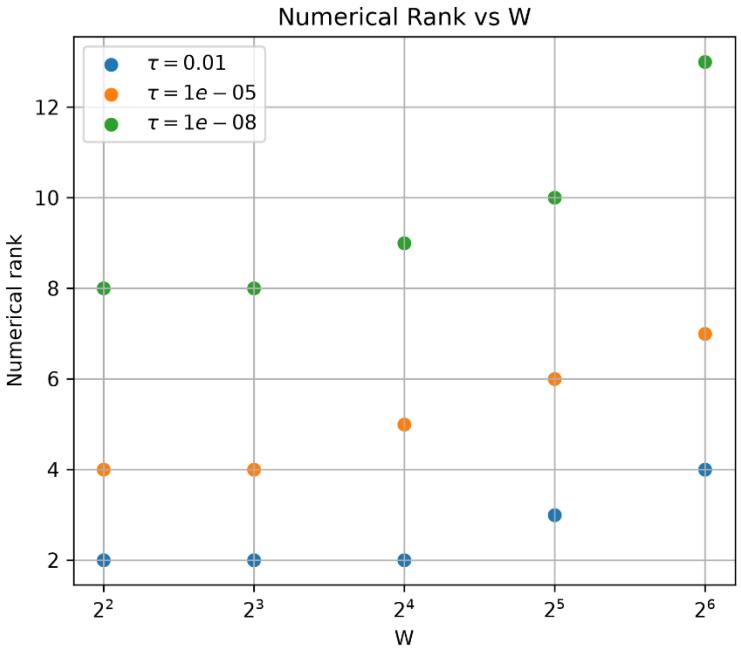


Figure 10 - Numerical rank vs W for different thresholds

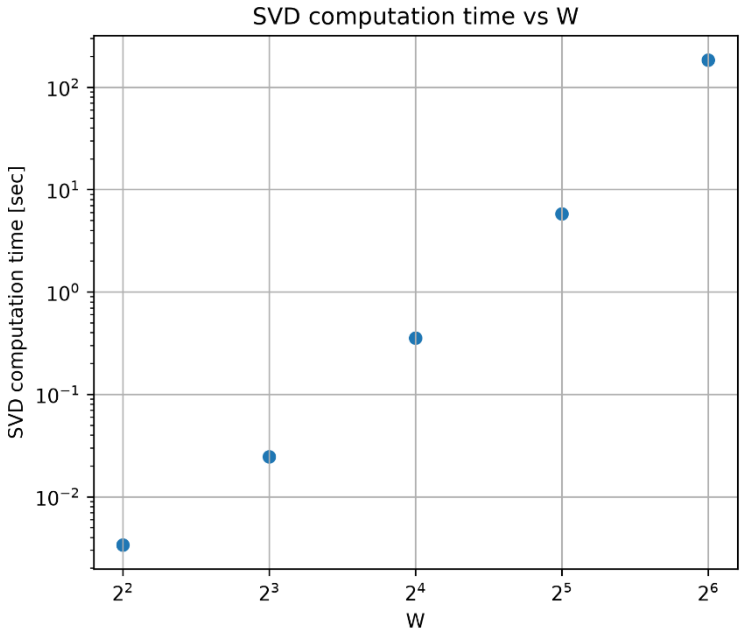


Figure 11 - SVD computation time vs W [sec]

Compared to Section (b), the oscillatory kernel produces a slower singular value decay. This can be seen in Figures 8 and 9, where more singular values remain significant before reaching the numerical precision region. The difference comes from the exponential phase term in the kernel, which adds oscillations depending on the distance between source and observer particles.

This behavior is also reflected in Figure 10. Unlike the non-oscillatory kernel, where the numerical rank stayed almost constant as W increased, here the numerical rank increases with W. Since λ is fixed, increasing W increases the physical size of the source and observer regions and introduces more oscillatory variation across the interaction block. Therefore, more singular values are needed to approximate A^{os} to the same tolerance.

Figure 11 shows that the SVD computation time increases sharply with W, similarly to Section (b). This increase is mainly caused by the growth in the size of A^{os}, since full SVD becomes expensive for larger dense matrices.

Section (d): Gemini

Prompt used:

I am working on a numerical linear algebra project about low-rank behavior of interaction matrix blocks between source and observer particles on a 2D grid.

In one experiment I used a non-oscillatory distance-based kernel. The domain size is W, and as I increased it, the normalized singular value curves were almost unchanged, and the numerical ranks for fixed relative tolerance (thresholds) stayed almost constant.

In the second experiment, I repeated the same setup with an oscillatory kernel that adds a complex phase depending on the distance between source and observer particles. In this case, the singular values decayed more slowly, and the numerical rank increased as W increased.

Can you explain why the oscillatory kernel leads to higher numerical rank and slower singular value decay compared to the non-oscillatory kernel?

Link to conversation:

https://gemini.google.com/share/ac4a7462741f

Summary:

Gemini explained that the non-oscillatory kernel used in Section (b) is smooth over well-separated source and observer regions. Because of this smoothness, the interaction block can be represented efficiently by a small number of dominant singular directions. When the entire geometry is scaled, the source and observer regions grow together with their separation, so the relative geometry remains almost the same. This explains why the normalized singular value curves remain similar and why the numerical rank stays almost constant as W increases.

For the oscillatory kernel in Section (c), Gemini explained that the additional phase term introduces oscillations that depend on the distance between source and observer particles. Since the wavelength is fixed, increasing W makes the source and observer regions contain more oscillatory variation. To approximate this interaction accurately, more singular vectors are needed to represent these

variations. Therefore, the singular values decay more slowly and the numerical rank increases as W increases.

Critical Evaluation:

Gemini's explanation was useful because its main idea matched the numerical behavior observed in the experiments. In Section (b), the relative singular value curves remain almost unchanged as W increases, and the numerical ranks remain constant for the tested thresholds. This supports the explanation that the non-oscillatory kernel is smooth and mainly controlled by the relative geometry of the two separated regions. In Section (c), the singular values decay more slowly and the numerical rank increases with W, which agrees with the explanation that the oscillatory phase introduces additional spatial variation when the physical domain grows while the wavelength remains fixed.

However, Gemini also expanded into topics that were beyond the scope of our specific question, such as Fast Multipole Methods, H-matrices, butterfly factorizations, and asymptotic rank scaling. These concepts may be related to the broader field, but they were not part of the experiments I performed in this section, and I have not studied them deeply enough to verify them independently. Therefore, I treated those parts of the response only as additional background, not as conclusions for the project. The parts I used in my analysis are the ones that directly match the numerical results shown in my singular value and rank plots.

Part II – Fast Rank Estimation

Section (e):

The algorithm tries to estimate the numerical rank of A^{os} using smaller sub-matrices.

The algorithm works like this:

1) Initialize the iteration $l = 1$ and the number $n = 10$, n is the number of unique rows and columns we take each iteration.
2) Randomly select n unique indices from 1 to N for both the rows and columns and naming these index vectors $\boldsymbol{i}_{row}, \boldsymbol{i}_{col}$ respectively.
3) Take the rows and columns corresponding to these index vectors and extract the interaction submatrix $A_l^{os} = A^{os}(\boldsymbol{i}_{row}, \boldsymbol{i}_{col})$.
4) Calculate the SVD of the submatrix and its numerical rank $R_l(\tau)$, for a given threshold τ.
5) If we are not in the first iteration, we compute the relative change in the numerical rank between the iterations, and if the relative change is smaller than some threshold τ_R, we conclude that the rank stabilized and we terminate and return $R_l(\tau)$.
6) Else, we go to the next iteration ($l \mathrel{+}= 1$), and we double n, $n \mathrel{*}= 2$, and return to step 2.

As for ε_R and τ_R: ε_R denotes the relative change in the numerical rank between iterations, that means that it represents how much the numerical rank changed if we take double the rows and columns, so when this value do not change much with the iteration, we conclude that we start to converge and using the amount of rows and columns we use, we can extract the numerical rank of the matrix A^{os} relatively well.

τ_R represent the tolerance we are willing to accept between each iteration, if the numerical rank does not change enough with the iteration, meaning $\varepsilon_R < \tau_R$, the change is too small, and we conclude that we are close enough to convergence.

For this algorithm to work, we require the numerical rank structure of A^{os} to be captured by sufficiently large random submatrices. This means that the important rank information should not be concentrated in only a few special rows or

columns. In that case, random row and column samples can capture the dominant directions of the matrix, and when the rank estimate stabilizes, we conclude that it is close enough to the numerical rank of the full matrix A^{os}.

Because this is a random estimation, the results may vary between random draws.

Section (f):

In this section we repeat the rank study from Section (c) using the oscillatory kernel while using the fast rank estimation algorithm to compare it to the full SVD rank from part I and pushing to larger W values without constructing the full matrix A^{os} and to find the scaling of the rank as W increases.

We have used the same variables as in Section (c) and the same thresholds, while for the stopping criterion of the fast algorithm, we used $\tau_R = 0.05$, this means that the algorithm stops when the relative change in the estimated rank of A^{os} between consecutive iterations is less than 5%.

For the comparison between the full SVD and fast rank we pushed up to $W = 64$, while for the fast rank alone we pushed up to $W = 1024$.

The following figure shows the performance of the fast rank algorithm compared to the full SVD:

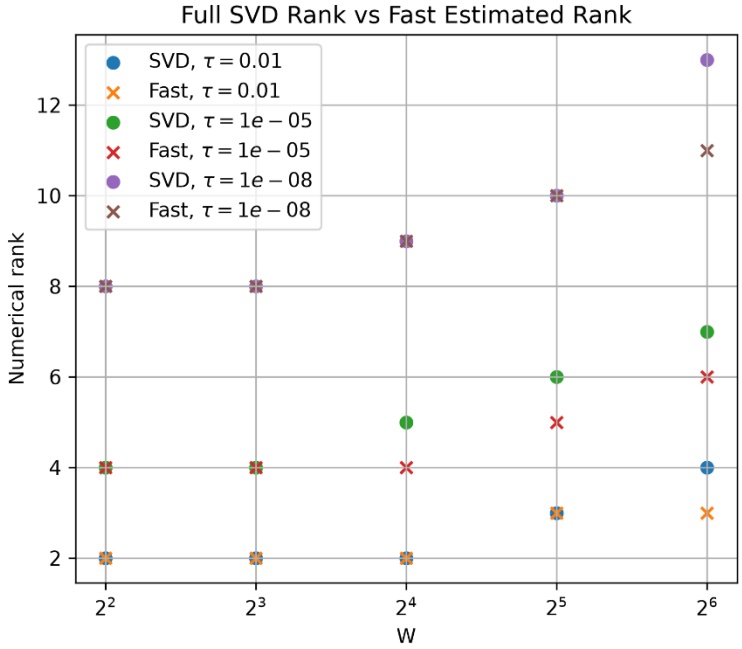


Figure 12 – Full SVD rank vs Fast Estimated rank

We can see that the fast algorithm ranks are close to the full SVD ranks for all the different thresholds. There are some differences, which are expected because the fast algorithm uses randomly sampled rows and columns and stops according to the stabilization tolerance τ_R. Overall, we see that the algorithm captures the trend of the full SVD ranks, therefore we conclude that we can use this algorithm for estimating the behavior of the ranks for larger W values.

As for the second part of this section, we pushed W to larger values and to find the asymptotic scaling of the rank with W. The following figure shows the rank versus W for the fast algorithm:

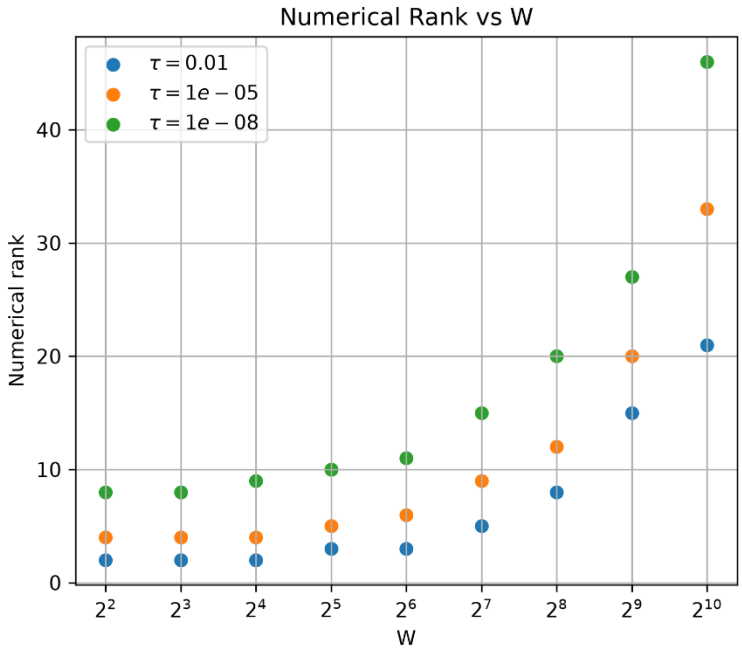


Figure 13 - Rank vs W for different thresholds for fast algorithm

From the figure we can see how the numerical rank of the fast algorithm increases versus W for larger values of W which was not feasible for the full SVD.

Next, we calculated the asymptotic scaling of the numerical rank with W for the different thresholds and obtained the following results:

τ	C	p	$R(W) \sim CW^p$
10^{-2}	0.15	0.72	$R(W) \sim 0.15W^{0.72}$
10^{-5}	0.46	0.61	$R(W) \sim 0.46W^{0.61}$
10^{-8}	1.33	0.50	$R(W) \sim 1.33W^{0.50}$

Table 1 - Asymptotic scaling of the numerical rank with W

The scaling was computed by fitting the numerical ranks to a power law of the form $R(W) \sim CW^p$.

The fit was performed using the values $W \geq 64$ to avoid the pre-asymptotic behavior of the smaller W values.

For all three thresholds, the fitted exponent satisfies $p < 1$. This means that the numerical rank grows sublinearly with W. In other words, when W is doubled, the numerical rank increases, but by less than a factor of two. We also see that smaller values of τ give larger numerical ranks, because more singular values are counted.

This result is consistent with Figure 13. As W increases, the oscillatory kernel contains more phase variation across the source and observer regions, so more singular modes are needed to approximate the interaction matrix. Therefore, the numerical rank increases. However, the growth is still sublinear because the interaction matrix remains structured and compressible. The kernel depends on the distance between separated source and observer regions, so many rows and columns are still correlated.

To conclude this section, the fast rank-estimation algorithm shows that the numerical rank of the oscillatory interaction block increases with W, but much more slowly than the full matrix size.

Section (g): ChatGPT

Prompt used:

I am working on a numerical linear algebra project involving the numerical rank of an observer-source interaction matrix for an oscillatory kernel.

In the experiment, the physical domain size W is repeatedly doubled. For smaller W values, I compared the rank estimated by a randomized sampled-submatrix algorithm to the rank computed by the full SVD, and the two were close. For larger W values, the full matrix and full SVD became too expensive, so I used only the fast sampled-submatrix estimator.

The plot shows numerical rank versus W for three different relative accuracy thresholds. For all thresholds, the numerical rank increases as W increases, but the growth is slower than linear. In other words, when W doubles, the rank increases, but usually by less than a factor of two. For a fixed W, smaller values of tau give larger numerical ranks, because the cutoff is lower and therefore more singular values are counted. However, for all tested thresholds, the rank growth with W appears sublinear.

Why does the numerical rank scale this way for an oscillatory kernel? Please explain intuitively, focusing on the interaction between oscillation, distance, smoothness, and low-rank compressibility.

Link to conversation:

https://chatgpt.com/share/6a426c6c-3d78-83eb-83c5-ac4e254f7c24

Summary:

The AI explained that the numerical rank represents the number of independent interaction patterns needed to describe the observer-source matrix. For the oscillatory kernel from Section (c), increasing W increases the amount of phase variation across the source and observer regions, so more singular modes are needed and the numerical rank increases.

However, the AI also explained that the rank does not grow linearly because the matrix is still structured. The kernel depends on distance, and for separated source

and observer regions this dependence remains smooth and correlated. Therefore, many rows and columns are not independent, and the matrix remains compressible. This explains why the rank grows with W, but sublinearly. The AI also noted that smaller τ values give larger numerical ranks because more singular values are counted.

Critical Evaluation:

The AI response gives a reasonable qualitative explanation of the observed behavior. It correctly connects the rank growth to the increasing oscillatory phase variation as W grows, and it correctly explains that separation and smooth distance dependence preserve low-rank structure.

However, the answer remains general and does not prove the measured scaling exponent. It also used a generic oscillatory kernel form and did not exactly match the decay factor in this project, which is $1/\sqrt{|\,\boldsymbol{r}_m - \boldsymbol{r}_n\,|}$. Therefore, the AI explanation is useful for intuition, but the actual conclusion about sublinear rank growth should be based on the numerical results and the fitted scaling from Section (f).

Section (h):

In this section we analyze the computation time of the fast rank-estimation algorithm as a function of W and find its asymptotic scaling.

For each value of W, we measured the total time required to estimate the numerical ranks for all three thresholds used in Section (f): $\tau = 10^{-2}, 10^{-5}, 10^{-8}$. Since the algorithm is randomized and timing measurements can vary between runs, each value of W was tested 31 times, and the median runtime was used.

The asymptotic fit was performed using only $W \geq 64$, because the smaller values of W are more affected by constant overhead and do not represent the asymptotic region.

We obtained the following figure:

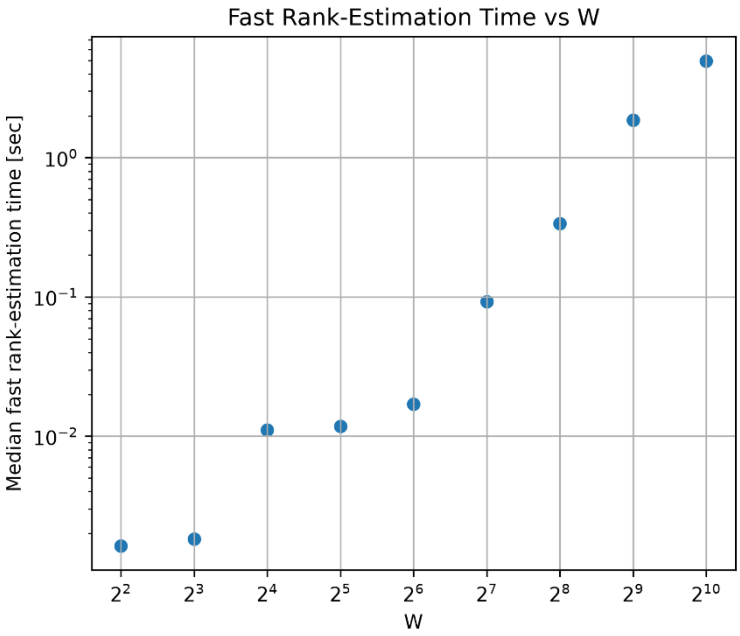


Figure 14 - Fast rank-estimation computation time vs W

The computation time was fitted to a power law of the form $T(W) \sim CW^{p}$, similar to Section (f).

The obtained scaling was:

$$T(W) \sim 3.54 \cdot 10^{-6} \times W^{2.07}$$

Therefore, the measured time scaling is approximately quadratic:

$$T(W) \sim W^{2}$$

This result is consistent with the expected cost of the algorithm. At each iteration, the sampled matrix has size $n \times n$. Constructing this sampled matrix costs $O(n^2)$, while computing its SVD costs $O(n^3)$. For larger n, the SVD cost is the dominant part.

From Section (f), the numerical rank of the oscillatory kernel scaled approximately like $W^{0.5}$ to $W^{0.7}$. Since the final sampled size n is expected to grow roughly with the numerical rank, we expect the computation time to scale approximately like

$$T(W) \sim n^3 \sim W^q, \quad 1.5 \leq q \leq 2.1$$

The measured exponent $p = 2.07$ is inside this expected range, so the timing results agree with the theoretical behavior.

For the non-oscillatory kernel from Section (a), the behavior should be different. In Part I, the numerical rank for this kernel remained almost constant as W increased. Therefore, the fast rank-estimation algorithm should not need much larger sampled matrices as W grows. As a result, we expect the computation time for the non-oscillatory kernel to grow much more slowly than in the oscillatory case and possibly remain almost constant over the tested range.

Part III – Fast LR Approximation

Section (i): Informed Blocked Randomized LR approximation

The goal of this part is to approximate the interaction block A^{os}, denoted in the algorithm by A, using a low-rank factorization $A \approx \widehat{U}\widehat{B}$.

The algorithm is called informed because it first uses the fast rank-estimation method from Part II to estimate the numerical rank R_l. In the blocked version, this rank is divided into N_b blocks, so the block size is $B_0 = \frac{R_l}{N_b}$.

The algorithm initializes $\widehat{U}, \widehat{B}$ to be empty matrices, computes the initial $\|A\|_F$, and sets $l = 1$.

At each iteration, a Gaussian random matrix $G \in \mathbb{C}^{N\times(B_0+\gamma)}$ is generated, and the matrix is sampled by $M = AG$.

Since M contains random linear combinations of the columns of A, it samples the range of the current matrix.

We then compute the SVD: $U\Sigma V^* = M$, and the first B_0 columns of U are used as a basis for the main directions captured in this block.

The corresponding coefficient block is $B := U^*(:,1:B_0)A$.

Then $U(:,1:\mathcal{B}_0)B$ is the low-rank block extracted from the current matrix. The factors are updated by

$$\widehat{U} = \left[\widehat{U}\ \ U(:,1:\mathcal{B}_0)\right], \qquad \widehat{B} = \begin{bmatrix}\widehat{B}\\ B\end{bmatrix}.$$

Meaning that at each iteration, we add B_0 captured directions vectors to $\widehat{U}$, and add the corresponding B_0 coefficient block rows to $\widehat{B}$.

The algorithm stops when

$$\|B\|_F \le \tau\|A\|_F$$

meaning that the newly extracted block is small relative to the original matrix norm. Otherwise, the residual is updated by

$$A = A - U(:,1:\mathcal{B}_0)B$$

Then, the next iteration samples the remaining part of the matrix.

As for the parameters, the parameter τ controls the required accuracy through the Frobenius-norm stopping criterion. Smaller τ usually gives more iterations, larger final rank, and better approximation.

The parameter γ is the oversampling parameter, increasing it improves the probability of capturing important directions through the Gaussian matrix G, but at increased cost.

The parameter N_b controls the block size, larger N_b gives smaller blocks and more residual updates, which can improve adaptivity but adds overhead.

As for the computational complexity, for $A \in \mathbb{C}^{N\times N}$, and suppose the algorithm performs l blocks, so that the final rank is $r = lB_0$.

In each block, forming $M = AG$ costs $O(N^2(B_0 + \gamma))$. The SVD of the thin matrix $M \in \mathbb{C}^{N\times(B_0+\gamma)}$costs $O(N(B_0 + \gamma)^2)$. Computing $B = U^*(:,1:B_0)A$ and updating the residual $A = A - U(:,1:B_0)B$ each cost $O(N^2B_0)$. Therefore, the leading total cost over all blocks is approximately

$$O(N^2(r + l\gamma)).$$

If the algorithm stops after approximately N_b blocks, then $r \approx R_l$, and the complexity becomes:

$$O(N^2(R_l + N_b\gamma))$$

This is cheaper than the full SVD cost $O(N^3)$ when $r + l\gamma \ll N$. Therefore, the algorithm is efficient when the interaction block is numerically low rank, and the oversampling and number of blocks remain moderate.

Section (j):

In this section, we implemented the informed blocked randomized LR approximation algorithm from Section (i), using the same oscillatory kernel and source-observer configuration from Section (c). The tested thresholds were

$$\tau = 10^{-2}, 10^{-5}, 10^{-8}.$$

For each value of W, we computed the full SVD of A^{os} and compared it to the fast LR approximation $A^{os} \approx \hat{U}\hat{B}$.

The approximation error of the fast LR method was measured by:

$$E_{LR} = \frac{\left\| A^{os} - \hat{U}\hat{B} \right\|_F}{\|A^{os}\|_F}$$

For comparison, we used the best SVD approximation with the same final rank r_{LR} produced by the fast LR algorithm. Its relative Frobenius error is:

$$E_{SVD,r_{LR}} = \frac{\left(\sum_{i=r_{LR}+1}^{N} \sigma_i^2\right)^{\frac{1}{2}}}{\left(\sum_{i=1}^{N} \sigma_i^2\right)^{\frac{1}{2}}}$$

This is the best possible rank r_{LR} approximation error in the Frobenius norm, so it is the natural benchmark for the fast LR algorithm.

The relative difference between the fast LR approximation and the best SVD approximation was computed by:

$$\frac{E_{LR} - E_{SVD,r_{LR}}}{E_{SVD,r_{LR}}}$$

To choose the algorithm parameters, we tested several values of γ and N_b at $W = 64$ and $\tau = 10^{-5}$ where $r_{SVD} = 7$. The results were:

γ	N_b	r_{LR}	Blocks	LR time [sec]	E_{LR}	Relative Difference
1	2	12	3	2.47	3.77×10^{-8}	1.69
5	2	12	3	2.43	2.48×10^{-8}	0.77
10	2	12	3	2.40	1.89×10^{-8}	0.35
20	2	12	3	2.64	1.81×10^{-8}	0.29
10	1	14	2	1.68	3.77×10^{-9}	0.80
10	5	10	5	3.98	1.42×10^{-7}	0.75
10	10	8	8	10.95	8.48×10^{-7}	0.34

Table 2 – Parameter calibration for the LR approximation algorithm at $W = 64$ and $\tau = 10^{-5}$

From the table, increasing γ improves the approximation error, because the random sampling captures the important residual directions more reliably. However, the improvement from $\gamma = 10$ to $\gamma = 20$ is small, while $\gamma = 20$ uses a larger sampled matrix in each block, and takes longer. Therefore, $\gamma = 10$ was chosen as a sufficient oversampling value. For N_b, larger values give smaller blocks and more residual updates, but they also increase the overhead. The case $N_b = 10$ was much slower, while $N_b = 2$ gave a good balance between runtime, final rank, and accuracy. Therefore, for the final comparison we used

$$\gamma = 10, N_b = 2.$$

Next, we look at the computation times of the fast LR method with the full SVD.

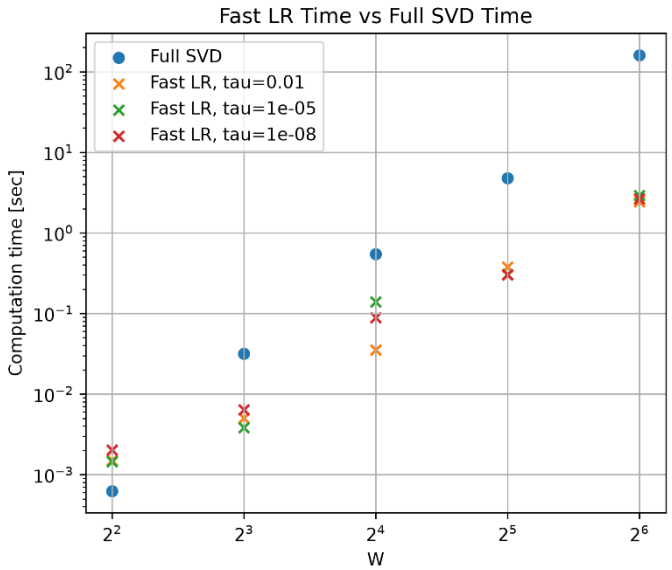


Figure 15 - Fast LR time vs full SVD time [seconds]

As can be seen in Figure 15, for small W, the full SVD is already cheap, so the advantage of the fast method is limited. However, for larger W, the difference becomes significant. At $W = 64$, the full SVD took approximately 160 seconds while the fast LR method took approximately $2.44 - 2.90$ seconds depending on the threshold. This shows that the fast LR method is much faster than the full SVD for the largest tested matrix.

Figure 16 compares E_{LR} with $E_{\mathrm{SVD},r_{\mathrm{LR}}}$:

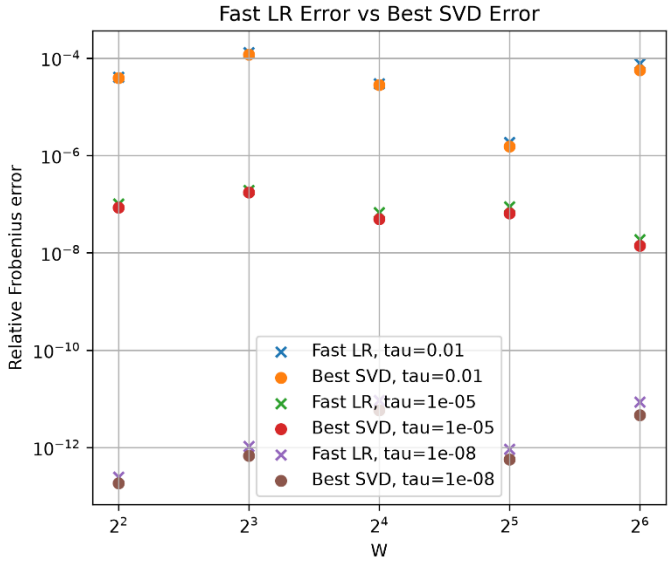


Figure 16 - Relative Frobenius error of Fast LR vs best SVD

As expected, the SVD error is smaller because the truncated SVD gives the best rank r_{LR} approximation. However, the fast LR error remains close to the SVD benchmark. For example, at $W = 64$ and $\tau = 10^{-5}$, we get $E_{\mathrm{LR}} = 1.89 \cdot 10^{-8}$, while $E_{\mathrm{SVD},r_{\mathrm{LR}}} = 1.400 \cdot 10^{-8}$.

The relative difference in this case is approximately 0.35.

The final estimated low rank is not always equal to the SVD numerical rank. For example, at $W = 64$ and $\tau = 10^{-5}$, the SVD numerical rank is 7, while the LR approximation uses rank 12. This is expected because the blocked algorithm adds whole blocks of size

$$B_0 = \left\lceil \frac{R_l}{N_b} \right\rceil$$

and stops only after the newly extracted block becomes small enough.

Overall, the informed blocked randomized LR approximation gives a large speedup compared to the full SVD, while keeping the approximation error close to the best SVD approximation at the same final rank. The parameters

$$\gamma = 10, N_b = 2$$

gave a balanced choice between accuracy, final rank, and runtime for the tested matrices.

Section (k):

In this section we use the fast LR approximation algorithm with the parameters selected in Section (j):

$$\gamma = 10, N_b = 2$$

The algorithm was tested for both the non-oscillatory kernel from Section (b), and the oscillatory kernel from Section (c).

For each kernel we used the same thresholds:

$$\tau = 10^{-2}, 10^{-5}, 10^{-8}$$

For each value of W, the fast LR algorithm was run 5 times, and the median runtime was used in order to reduce timing noise. The final LR rank r_{LR} was also recorded.

Although the question asks for the scaling with N, we plot the results as a function of W, similarly to the previous sections. This is equivalent for studying the trend, since the number of particles in each source/observer block is approximately:

$$N \sim W^2$$

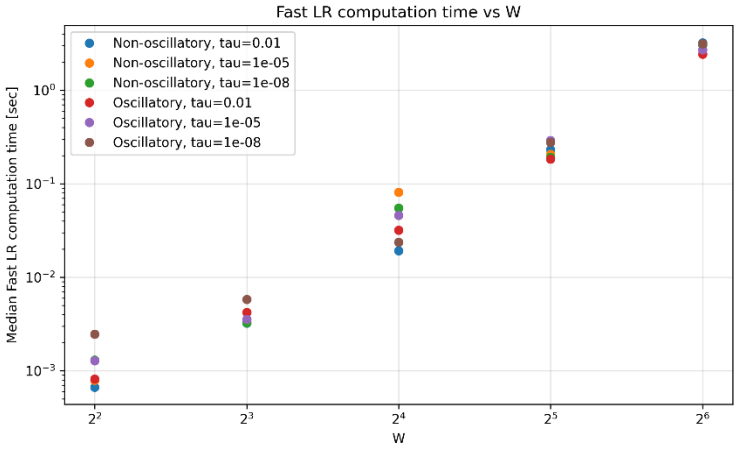


Figure 17 – Median Fast LR computation time vs W for different thresholds and kernels [sec]

Figure 17 shows the median computation time of the fast LR method as a function of W. For both kernels, the runtime increases as W increases. This is expected because the matrices A^{os} become larger, and the algorithm still performs dense matrix multiplications and residual updates.

At $W = 64$, the fast LR method takes only a few seconds, while the full SVD times from Sections (b) and (c) are much larger. To compare the methods more directly, we used the full SVD results as a reference and computed the speedup of fast LR over full SVD for the largest tested matrix.

Kernel	τ	SVD time [sec]	Fast LR time [sec]	Speedup	SVD rank	LR rank
Non-Oscillatory	10^{-2}	73.10	3.23	22.63	1	2
Non-Oscillatory	10^{-5}	73.10	3.09	23.68	3	6
Non-Oscillatory	10^{-8}	73.10	3.05	23.99	6	9
Oscillatory	10^{-2}	201.24	2.43	82.67	4	6
Oscillatory	10^{-5}	201.24	2.71	74.24	7	12
Oscillatory	10^{-8}	201.24	3.15	63.96	13	21

Table 3 – Full SVD and Fast LR comparison at $W = 64$

From the table, we can see that for $W = 64$ the fast LR method is much faster than the full SVD for both kernels. For the non-oscillatory kernel, the speedup remains steady at around $23\times$, while for the oscillatory kernel, the speedups are between approximately $64\times$ and $83\times$. Therefore, even though the fast LR method still uses dense matrix operations, it is significantly cheaper than computing the full SVD.

Next, we fitted the fast LR computation time to a power law of the form

$$T(W) \sim CW^p$$

Similar to previous sections.

The fitted exponents we obtained are in the following table:

Kernel	τ	p
Non-oscillatory	10^{-2}	3.35
Non-oscillatory	10^{-5}	3.10
Non-oscillatory	10^{-8}	3.14
Oscillatory	10^{-2}	3.00
Oscillatory	10^{-5}	3.14
Oscillatory	10^{-8}	3.08

Table 4 – Fitted exponents $T(W) \sim CW^p$ for the Fast LR method

The measured exponents are around $p \approx 3$. From Section (i), the leading cost of the fast LR method is approximately

$$O(N^2(R_l + N_b\gamma)).$$

Since in this two-dimensional geometry

$$N \sim W^2,$$

if $R_l + N_b\gamma$ is roughly constant, the theoretical leading cost corresponds to approximately

$$O(W^4).$$

This estimate is most relevant for the non-oscillatory kernel, where the rank remains approximately constant. For the oscillatory kernel, R_l increases with W, so the theoretical cost may grow faster. However, the measured exponents are lower than 4. This is not unexpected, because the tested range is finite and the runtime is affected by constant overhead, memory movement, and optimized dense matrix operations. Therefore, the fitted exponent should be interpreted as an empirical scaling over the tested range, not as the exact asymptotic complexity.

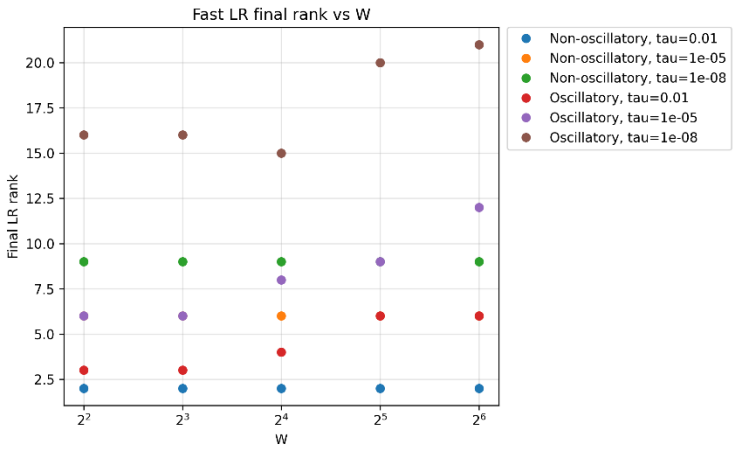


Figure 18 – Fast LR rank vs W for different thresholds and kernels

Figure 18 shows the final LR rank as a function of W. This plot shows the difference between the two kernels more clearly than the time plot. For the non-oscillatory kernel, the final LR rank remains constant as W increases:

$$r_{LR} = 2, 6, 9$$

For

$$\tau = 10^{-2}, 10^{-5}, 10^{-8}$$

Respectively. This agrees with Section (b), where the SVD numerical rank also remained almost constant with W. The LR ranks are larger than the SVD numerical ranks because the blocked algorithm adds whole blocks of size B_0. In addition, the algorithm stops only after a newly extracted block becomes small enough, so the final LR rank can exceed the SVD numerical rank for the same threshold.

For the oscillatory kernel, the final LR rank increases with W. At $W = 64$, the final LR ranks are

$$r_{LR} = 6, 12, 21$$

For

$$\tau = 10^{-2}, 10^{-5}, 10^{-8}$$

This agrees with Section (c), where the oscillatory kernel produced slower singular value decay and increasing numerical rank. The reason is that increasing W increases the physical size of the source and observer regions while the wavelength remains fixed, so the interaction block contains more oscillatory variations.

Overall, the fast LR method reproduces the qualitative behavior observed with the full SVD. The non-oscillatory kernel remains strongly low rank, while the oscillatory kernel requires increasing ranks as W grows. At the same time, the fast LR method gives a large speedup compared with the full SVD, especially for the largest tested matrices.

Part IV – Hierarchical Matrix Compression

Section (l):

An H-matrix is a way to represent a large dense interaction matrix using a hierarchy of smaller blocks. The main idea is that even if the full matrix is not low-rank, many parts of it can still be compressed. In particular, blocks that describe interactions between far-away particle groups are often approximately low-rank, while blocks that describe nearby interactions are usually more complicated and should be stored directly.

This representation makes sense for particle interaction matrices because the behavior of the kernel depends strongly on distance. When two groups of particles are close to each other, the interaction can change significantly from one particle pair to another. Therefore, the corresponding matrix block is usually harder to approximate with only a few low-rank factors. On the other hand, when two groups of particles are well separated, the interaction becomes smoother and more redundant. In that case, many rows or columns of the block contain similar information, so the whole block can often be approximated accurately using a low-rank representation.

The hierarchical structure is built by recursively dividing the physical domain into smaller square clusters. Since the particles are placed on a two-dimensional grid, a natural approach is to split the square domain into smaller square subdomains. Each cluster contains a subset of particles, and each matrix block represents the interaction between two such clusters. For every pair of clusters, an admissibility criterion is used to decide whether the corresponding block should be compressed or split further.

There are two common types of admissibility criteria: strong admissibility and weak admissibility. Strong admissibility uses the geometry of the clusters. A block is compressed only if the distance between the two clusters is large enough compared with their sizes. This criterion is more conservative, but it is also more reliable, because it only compresses blocks where low-rank behavior is expected.

Weak admissibility is less strict. In many cases, it treats off-diagonal blocks as admissible simply because the two clusters are different. This gives a simpler block structure and may reduce the amount of recursive splitting. However, it may also try to compress blocks whose clusters are close to each other. Such blocks may not be truly low-rank, so they may require higher ranks to approximate accurately.

Based on the results from kernels (b) and (c), I expect the H-matrix approach to be useful for both kernels, but more effective for kernel (b). For the non-oscillatory kernel in (b), the far-field blocks showed strong low-rank behavior. Therefore, many admissible blocks should be compressed using relatively small ranks, leading to significant memory savings.

For the oscillatory kernel in (c), the method should still be useful, but the compression is expected to be weaker. The oscillatory phase adds more variation to the matrix blocks, so higher ranks are needed to reach the same accuracy. Therefore, kernel (c) should still benefit from the H-matrix representation, but the memory savings are expected to be smaller than for kernel (b). In this case, strong admissibility is especially important, because it avoids compressing blocks that are not sufficiently separated.

Section (m): Strong Admissibility

Prompts used:

1) I am working on my Numerical Linear Algebra final project about hierarchical matrix compression.

We have particles placed on a 2D square grid inside a square domain.

The assignment asks me to build a data structure that represents a quadratic hierarchical tree of the points in S, partitioned based on their geometric location within a hierarchy of squared domains.

Based on this tree, I also need a routine that builds a hierarchical matrix data structure as a list of hierarchical matrix blocks. For each final block, the list entry should include:

- the participating row indices,

- the participating column indices,

- the number of row indices,

- the number of column indices,

- and an indicator of whether the block is admissible or not.

For my case, please use a strong admissibility criterion.

Can you help me understand the structure and write a simple Python implementation for this? Please keep the implementation simple and explain the main steps.

2) This looks good. Can you add simple verification checks for this structure?

I want to check:

1. that all particles appear exactly once in the leaves of the quadtree,

2. that the final block list covers the full matrix, meaning the sum of n_rows * n_cols over all blocks equals N^2,

3. that every block marked admissible really satisfies the strong admissibility criterion,

4. and that diagonal/self-interaction blocks are not marked admissible.

Please keep the checks simple and Python-oriented.

3) This is helpful. I want to keep the final block entries exactly as required by the assignment: row indices, column indices, number of rows, number of columns, and admissible true/false.

Can you explain how to use row_bounds and col_bounds only internally for verification/debugging, without presenting them as part of the required final block list? Also, please show a clean final version of the data structures, and all the verifying of the code as a final viable code for this section.

Summary of AI response:
The AI suggested building the structure in two stages. First, I construct a quadtree for the particles in S. The root node represents the full square domain, and each node is recursively divided into four smaller square subdomains according to the particle locations. Each node stores the particle indices inside the square, the square bounds, its level, and its children.

Then, based on this tree, I build the hierarchical matrix block list recursively from pairs of tree nodes. Each pair represents one matrix block. The first node gives the row indices, and the second node gives the column indices.

For my case, I used strong admissibility. A block is marked admissible if

$$\max(\mathrm{diam}(B_r), \mathrm{diam}(B_c)) \leq \eta \times \mathrm{dist}(B_r, B_c).$$

If this condition is satisfied, the block is stored as a final admissible block. If not, the block is split into smaller cluster pairs, unless both clusters are already leaves. In that case, the block is stored as a final non-admissible block.

The final block list contains only the fields required by the assignment:

$$(\text{row indices}, \text{column indices}, n_{\text{rows}}, n_{\text{cols}}, \text{admissible}).$$

The row and column bounds are used only internally for checking the admissibility condition, and are not part of the final block list.

Verification and correction:
To verify the implementation, I checked that every particle appears exactly once in the leaves of the quadtree. I also checked that the final block list covers the full matrix by verifying that

$$\sum_k n_{\text{rows},k} \times n_{\text{cols},k} = N^2.$$

For small test cases, I also checked exact matrix coverage, meaning that every matrix entry is covered by exactly one final block. In addition, I verified that every admissible block satisfies the strong admissibility criterion and that diagonal/self-interaction blocks are not marked as admissible.

The main correction I made to the AI response was to keep the final block entries minimal. The AI suggested using extra geometric information for debugging, but I kept this information only internally because the assignment only requires the row indices, column indices, their numbers, and the admissibility indicator.

Comment on AI response:
The AI response was useful because it gave a clear recursive structure for building the quadtree and the block list. However, I still had to check that the suggested structure matched the exact assignment requirements and did not include unnecessary fields in the final block representation.

Section (n):

In this section, we used the hierarchical block structure from Section (m) to estimate the memory required for storing the full interaction matrix A in H-matrix format.

For the full dense matrix, the memory is

$$M_{\text{dense}} = N^2$$

where N is the number of particles. For the H-matrix, non-admissible blocks are stored densely, so a block of size $n_i \times n_j$ requires $n_i \times n_j$ stored scalar values. Admissible blocks are stored in low-rank form. For each admissible block, we computed the singular values and chose the numerical rank r according to

$$r = R(\tau) = \#\left\{i : \frac{\sigma_i}{\sigma_1} > \tau\right\}$$

The memory of an admissible block is then $r(n_i + n_j)$.

Therefore, the total H-matrix memory is

$$M_H = \sum_{\text{dense blocks}} n_i \times n_j + \sum_{\text{admissible blocks}} r_k\,(n_i + n_j).$$

The compression ratio is defined as

$$\frac{M_H}{N^2}.$$

Memory is measured here as the number of stored scalar values.

The experiments were performed for both kernels from Sections (b) and (c), for

$$W = 1,2,4,8, \tau = 10^{-2}, 10^{-5}, 10^{-8}.$$

The hierarchy depth was tested using max level $= 2,3,4,5$, and the strong-admissibility parameter was tested using $\eta = 0.5,1,2$.

For the representative case $W = 8,\ \tau = 10^{-5}$, the best tested hierarchy depth was max level 4. Increasing the depth from 2 to 4 improved compression, but level 5 increased the number of blocks too much and slightly worsened memory. Among the tested admissibility parameters, $\eta = 2$ gave the best compression.

η	Best max level	Non-oscillatory compression ratio	Oscillatory kernel compression ratio
0.5	4	0.264	0.319
1	4	0.185	0.222
2	4	0.125	0.151

Table 5 – Parameter selection at $W = 8, \tau = 10^{-5}$

Therefore, among the tested values, the selected parameters are:

$$\tau = 10^{-5}, \eta = 2, maximum\ level = 4$$

Using these parameters, the H-matrix memory was compared to the full dense matrix as can be seen in Figure 19:

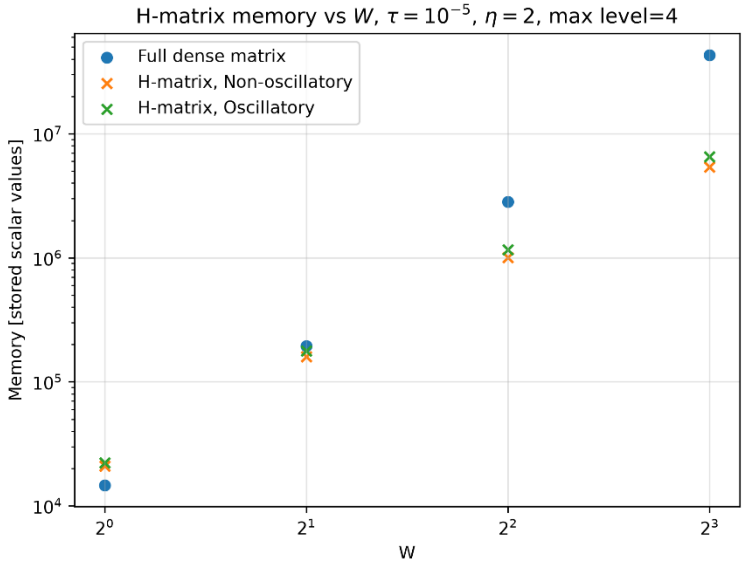


Figure 19 – H-matrix memory vs $W, \tau = 10^{-5}, \eta = 2, maximum\ level = 4$

Figure 20 shows the compression ratio:

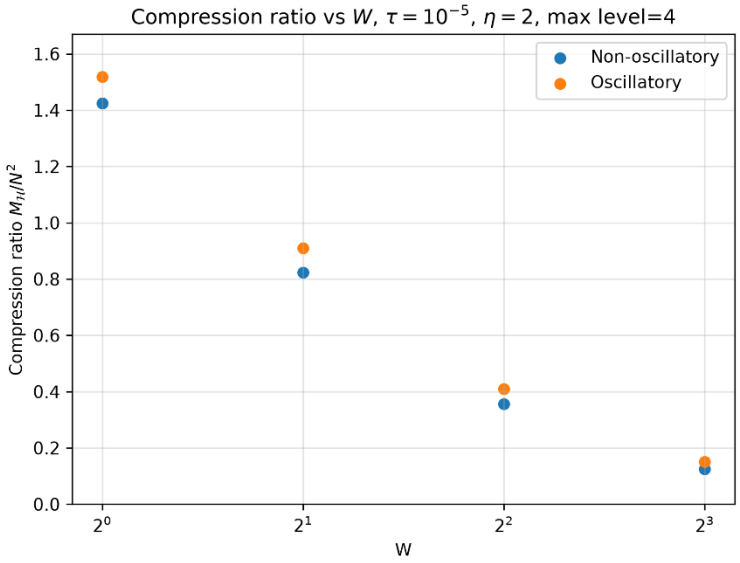


Figure 20 – Compression ratio vs W, $\tau = 10^{-5}, \eta = 2, maximum\ level = 4$

At $W = 8$, the full dense matrix requires $N^2 \approx 43\ million$ stored scalar values. Table 6 shows the resulting H-matrix storage.

Kernel	Dense memory N^2	M_H	Compression ratio	Average rank
Non-oscillatory	43,046,721	5,381,587	0.125	6.00
Oscillatory	43,046,721	6,516,563	0.151	7.31

Table 6 – Final memory savings at $W = 8$, $\tau = 10^{-5}, \eta = 2, maximum\ level = 4$

We see that for $W = 8$, the H-matrix stores only about 12.5% of the dense memory for the non-oscillatory kernel and about 15.1% for the oscillatory kernel.

For small W, the H-matrix may require more storage than the dense matrix because the blocks are small and the low-rank representation $r(n_i + n_j)$ can be larger than dense storage $n_i \times n_j$. For larger W, the far-field low-rank structure dominates and the compression becomes effective.

The oscillatory kernel required more memory than the non-oscillatory kernel. This is expected because the oscillatory phase creates more variation inside admissible blocks, leading to larger numerical ranks, as also observed in previous parts.

Finally, for the selected parameters, the fitted memory scaling for the non-oscillatory kernel was approximately:

$$M_H(W) \sim W^{2.54}$$

For the oscillatory kernel it was:

$$M_H(W) \sim W^{2.60}$$

Since $N \sim W^2$, its storage scales as:

$$M_{dense} = N^2 \sim W^4$$

Therefore, the H-matrix memory grows significantly more slowly than the dense matrix memory.

Appendix

AI Disclosure

Tools used – ChatGPT and Gemini

Purpose of use – General help in syntax for code, functions that helps with general python coding like function for saving figures and results for txt and for reproducibility sakes. As well as grammar and shortcuts in word and occasionally asking questions about misunderstood terminology and background. In addition, Gemini was used for the Part I Section (d).

For each question I created a chat to keep it clean, the following are links to the chats:

Part I – Preliminaries:

https://chatgpt.com/share/6a3c1c16-b600-83eb-8d5d-3027532ce9b8

Part II – Fast Rank Estimation

https://chatgpt.com/share/6a425fef-d270-83ed-9c57-15787db8c373

Part III – Fast LR Approximation

https://chatgpt.com/share/6a48e23c-e888-83eb-92db-4014228944f3

Part IV - Hierarchical Matrix Compression

https://chatgpt.com/share/6a4b49b5-8e54-83eb-8de0-00964e6a003a

https://chatgpt.com/share/6a4b49cb-12c8-83eb-9187-a1ffb573743b

These chats were used for more question-oriented questions, while I also used some small temporary chats aimed specifically at small debugging across the code.

For example:

“Give me a function to plot and save a figure”

“Give me a function to save a dict to a csv file”

All AI-assisted explanations were checked against the assignment instructions, the implemented code, and the numerical results obtained from my own runs.